UNIT-1

---1---

1> What is an Algorithm?

- An algorithm is just a step-by-step set of instructions to solve a problem.

 Example: A recipe for making tea is an algorithm (boil water → add tea → add sugar → add milk → done).

 Algorithm in ADA (Programming Language)

- In Ada, an algorithm is written using procedures or functions.
- You break the problem into steps.
- Each step is written as code.
- Ada follows structured programming, so the steps are very clear.

 Key Points

- Algorithm = steps to solve problem.
- Ada lets you write these steps in a clear, structured way.
- Focus on Input → Process → Output.

---2---

2> Key Characteristics of an Algorithm

1. **Finiteness**

 * Algorithm must **end after some steps**.

 * Example: “Add first 10 numbers” → ends after 10 steps.

2. **Definiteness (Clarity)**

 * Steps must be **clear and not confusing**.

 * Example: “Add 5 to number” is clear, but “Make number big” is not.

3. **Input**

 * Algorithm takes some **values to start**.

 * Example: To find the largest number → we input 2 numbers.

4. **Output**

 * Algorithm must **give result**.

 * Example: Largest number is shown as output.

5. **Effectiveness**

 * Steps must be **simple and possible to do**.

 * Example: “Multiply 2 numbers” is possible, but “Multiply till forever” is not.

6. **Generality**

 * Algorithm should work for **many cases, not just one**.

 * Example: Sorting algorithm works for any list of numbers, not just one list.

 In short (easy way):

* **It ends** (Finiteness)

* **It is clear** (Definiteness)

* **It takes input** (Input)

* **It gives result** (Output)

* **It is practical** (Effectiveness)

* **It works for many cases** (Generality)

---3---

3> Algorithms as a Technology

1. **What does it mean?**
   * Algorithm = step-by-step method to solve a problem.
   * As a technology → algorithms are used in **computers, apps, machines** to solve problems automatically.

---

2. **Why important?**
   * They make computers **think and act step by step**.
   * Help us solve problems **faster and correctly**.

---

3. **Examples in daily life:**
   * **Google Search** → Uses algorithms to show best results.
   * **Maps (GPS)** → Algorithm finds the shortest path.
   * **E-commerce (Amazon/Flipkart)** → Algorithm suggests products.
   * **Bank ATM** → Algorithm checks balance before giving money.

---

4. **Easy Summary:**
   * Algorithms are like the **"brain" of technology**.
   * They tell computers **what to do and how to do it**.
   * Without algorithms, computers are just machines with no clear instructions.

 One line:

Algorithms as a technology = using step-by-step methods to make computers and apps solve real-world problems automatically.

Perfect Let's break down **Designing Algorithms** into **easy and simple words**.

---4---

4> What is Designing Algorithms?

Designing an algorithm means **planning step-by-step instructions** to solve a problem in the **best way**.

 Just like writing a recipe before cooking.

Steps in Designing an Algorithm

1. **Understand the Problem**
   * First, know clearly what the problem is.
   * Example: "Find the largest number in a list."
2. **Decide Inputs and Outputs**
   * What information will go **in**? (Input)
   * What answer should come **out**? (Output)
   * Example: Input = list of numbers, Output = largest number.
3. **Break into Small Steps**
   * Write steps one by one, in order.
   * Example:

1. Read all numbers
2. Compare numbers
3. Keep track of the largest
4. Show result

4. **Choose the Best Method**
   * Think: Should I do it slowly (check one by one) or smartly (like divide and conquer)?
   * Example: Linear search vs Binary search.
5. **Write in Pseudocode or Flowchart**
   * Before real code, make it simple (like plain English or a diagram).
6. **Convert to Programming Language**
   * Finally, write in Ada, C, Python, or any language.

Example: Finding Largest of 3 Numbers

1. Start
2. Take 3 numbers as input
3. Compare them one by one
4. Keep the largest number
5. Print the result
6. End

Easy Summary

Designing an algorithm =

* **Understand problem → Plan inputs & outputs → Break into steps → Choose best method → Write & test.**

---5---

Introduction to Various Methods of Solving Algorithms

5> What does it mean?

There are many **ways (techniques/methods)** to design and solve algorithms.

Each method is like a “style” of solving a problem.

Memory Trick (Short Form)

“Baghho Dafod Gando Dafod Baghho Ran”

B → Brute Force

D → Divide and Conquer

G → Greedy

D → Dynamic Programming

B → Backtracking

R → Randomized

Common Methods (Easy Words + Examples)

1. **Brute Force (Simple Try-All Method)**
   * Try **all possible solutions** until you find the correct one.
   * Example: To find a number in a list → check one by one.
2. **Divide and Conquer**
   * Break the problem into **smaller parts**, solve them, then combine.

* Example: Binary Search (divide list into half each time).

* Example: Merge Sort.

3. **Greedy Method**

* At each step, **choose the best immediate option** (local best).

* Example: Choose shortest distance at each step in a map.

4. **Dynamic Programming**

* Solve problem by **breaking into smaller problems** and **storing results** so you don't repeat work.

* Example: Fibonacci numbers (store previous results).

5. **Backtracking**

* Try one option → if it doesn't work, **go back** and try another.

* Example: Solving a maze → if wrong path, go back and try new path.

6. **Randomized Algorithms**

* Use **random choices** to make the algorithm faster or simpler.

* Example: QuickSort (choose random pivot).

Easy Summary

* **Brute Force** → Try everything.

* **Divide & Conquer** → Break into parts.

* **Greedy** → Pick best choice at each step.

* **Dynamic Programming** → Save results, avoid repeating.

* **Backtracking** → Go back if wrong path.

* **Randomized** → Use random choices.

In short: Different methods = different ways of solving depending on the problem.

UNIT-2

---1---

1> Recurrence Relation

A **recurrence relation** is just a **formula** that says:

To solve a **big problem**, we solve **smaller problems of the same type**.

- It's used when an algorithm calls itself (recursion).
- It is used in recursive algorithms.

It's like: *today's answer depends on yesterday's answer.*

Example 1: Factorial (n!)

Factorial(5) = 5 × Factorial(4)

Factorial(4) = 4 × Factorial(3)

... until Factorial(1).

Each step depends on the **previous smaller step**.

Example 2: Fibonacci Numbers

F(n) = F(n-1) + F(n-2)

* To find the 5th number, you need the 4th and 3rd.

 Each number depends on **smaller numbers before it**.

Example 3: In Algorithms

* **Binary Search** → T(n) = T(n/2) + 1

  (cut list into half each time)

* **Merge Sort** → T(n) = 2T(n/2) + n

  (divide list into 2 halves, then merge).

Easy Summary in 1 Line

Recurrence relation = a formula where the solution of a problem comes from smaller versions of the same problem.

---2---

2> Why solve recurrence? & Methods

Because recurrence is a formula, but we need to know the time complexity (Big-O) of an algorithm.

Got it Let's make **Methods to Solve Recurrences** in the **easiest words possible**.

Methods to Solve Recurrences (Super Simple)

1. **Substitution Method (Guess & Check)**

* **Think like this:** Guess the answer (time complexity), then prove it.

* Example: T(n) = T(n-1) + 1

  * Guess = O(n)

  * Check step by step → correct.

 Like solving a puzzle by making a smart guess.

---

2. **Recursion Tree Method (Draw a Tree)**

* **Think like this:** Draw a tree of all recursive calls and add the work at each level.

* Example: Merge Sort → T(n) = 2T(n/2) + n

  * Each level does "n" work

  * Log n levels → total = n log n.

 Like cutting a big problem into pieces and adding all the work together.

---

3. **Master Method (Shortcut Formula)**

* **Think like this:** A ready-made formula to quickly solve recurrences of the form:

  T(n) = aT(n/b) + f(n)

* Steps:

  1. Find n^log_b(a).

  2. Compare f(n) with it.

  3. Apply 1 of 3 cases.

* Example: T(n) = 2T(n/2) + n

  * a=2, b=2 → $n^{\log_2 2} = n$

  * f(n) = n → same → O(n log n).

 Like a calculator shortcut to avoid long solving.

Easy Summary in 1 Line Each

* **Substitution** → Guess the answer & prove.

* **Recursion Tree** → Draw a tree & add work.

* **Master Method** → Use formula shortcut.

 Memory Trick: **“Silly Red Monkey”**

* **S** → Substitution

* **R** → Recursion Tree

* **M** → Master Method

---2---

2> Divide and Conquer

Idea:

* **Divide** → Break big problem into small parts.

* **Conquer** → Solve each small part.

* **Combine** → Join small answers to get the final answer.

Like cutting a big cake → eat small slices → enjoy whole cake.

Algorithms:

1. **Strassen’s Matrix Multiplication**

* **Goal:** Multiply big matrices faster.

* **Steps:**

  1. Break big matrix into 4 small parts.
  2. Multiply parts (only 7 multiplications needed).
  3. Add/subtract to get the final big matrix.

* **Why:** Faster than normal multiplication.

---

2. **Multiplying Two Large Numbers (Karatsuba)**

* **Goal:** Multiply very large numbers fast.

* **Steps:**

  1. Break numbers into halves.
  2. Multiply halves in a smart way (3 multiplications).
  3. Add results to get final number.

* **Why:** Much faster than normal multiplication.

---

3. **Binary Search**

* **Goal:** Find a number in a sorted list.

* **Steps:**

  1. Look at the middle number.
  2. If it’s the number → done.
  3. If number is smaller → check left half.
  4. If number is bigger → check right half.

* **Why:** Very fast, only looks at few numbers.

---

4. **Merge Sort**

* **Goal:** Sort numbers in order.

* **Steps:**

  1. Split list into 2 halves.
  2. Sort each half.
  3. Merge halves together to make one sorted list.

* **Why:** Always fast and works for big lists.

---

5. **Quick Sort**

* **Goal:** Sort numbers in order.

* **Steps:**

  1. Pick a pivot number.
  2. Make 2 groups: smaller than pivot, bigger than pivot.
  3. Sort both groups.
  4. Combine pivot and groups → sorted list.

* **Why:** Fast for most lists.

---

6. **Maximum Subarray Problem**

* **Goal:** Find part of array with largest sum.

* **Steps:**

  1. Split array into 2 halves.
  2. Find max sum in left half.
  3. Find max sum in right half.
  4. Find max sum crossing middle.
  5. Pick the biggest of the 3.

* **Why:** Faster than checking all parts one by one.

🧠 Super Easy Summary

Divide & Conquer = Break → Solve → Join

* **Matrix multiply** → break → multiply → join

* **Large number multiply** → break → multiply → join

* **Binary search** → break → check → pick half

* **Merge sort** → break → sort → merge

* **Quick sort** → pick pivot → split → sort → join

* **Max subarray** → split → find max → combine

---3---

3> 🔹 What is Dynamic Programming (DP)?

* **Dynamic Programming** is a way to solve problems **step by step** and **remember previous answers** so we don't repeat work.

* Think of it as **smart recursion with memory**.

*Dynamic Programming = Solve small parts once, save answers, use them to solve big problem faster.

🧠 Easy Example

1. **Fibonacci Numbers**

   * Normal recursion: F(n) = F(n-1) + F(n-2) → repeats calculations many times.

* DP: Store results of F(1), F(2), … so you use them instead of recalculating.

* Faster and saves time.

🔑 Key Idea

1. Break the problem into **small parts** (like recursion).

2. Solve each part **once**.

3. **Save the answer** (in array/table).

4. Use saved answers for bigger parts.

🧠 Steps to Solve DP Problems

1. **Define subproblems** → Break problem into small pieces.

2. **Find recurrence** → make formula to relate subproblems.

3. **Choose storage** → usually an array/table.

4. **Fill table** → solve subproblems one by one.

5. **Get final answer** → use table to find solution.

🎯 Easy Examples of DP Problems

* Fibonacci numbers

* Minimum coin change

* Maximum sum subarray

* Longest common subsequence

🚀 Easy Summary in 1 Line

Dynamic Programming = Solve small problems, save answers, reuse them → faster and smart solving.

🧠 Steps of DP (Easy Words)

1. Define subproblems → small pieces
2. Find recurrence → formula
3. Choose storage → array/table
4. Fill table → solve step by step
5. Get final answer → use table

🧠 Memory Trick Sentence

🎯 “DR(doctor) Set For Fun” 🚑🎉

D → Define subproblems

R → Recurrence

S → Storage

F → Fill table

F → Final answer

---4---

4> 1️⃣ **Coin Change Problem**

Problem:

* You have some coins. You want to make a total **amount using as few coins as possible**.

Example:

* Coins = 1, 2, 5; Amount = 11

* Answer = 5 + 5 + 1 → **3 coins**

Easy Idea:

- For each amount, check all coins and pick minimum coins needed.

Summary:

- Goal: Minimum coins
- Method: Top-Down or Bottom-Up
- Table/Array: Store minimum coins for each amount

2️⃣ **Rod Cutting Problem**

Problem:

* You have a rod and prices for each piece.

* Cut the rod in pieces to get **maximum money**.

Example:

* Rod length = 4, Prices = 1→2, 2→5, 3→7, 4→8

* Best way: 2 + 2 → 5 + 5 = **10 money**

Easy Idea:

- Try all possible cuts → choose cut giving max total money.

Summary:

- Goal: Maximum money
- Method: Top-Down or Bottom-Up
- Table/Array: Store max money for each rod length

3️⃣ **Longest Common Subsequence (LCS)**

Problem:

* Two sequences. Find the **longest sequence that appears in both**.

Example:

* X = “AGGTAB”, Y = “GXTXAYB”

* LCS = “GTAB” → length 4

Easy Idea:

- Compare letters → if same, include → else, take max of skipping one letter.

Summary:

- Goal: Longest common letters
- Method: Top-Down or Bottom-Up
- Table/Array: 2D table for prefixes of sequences

🔑 Super Easy Summary

| Problem | Easy Idea |

| ---------- | -- |

| Coin Change | Use fewest coins to make amount |

| Rod Cutting | Cut rod to get max money |

| LCS | Find longest common letters in both sequences |

🧠 Super Short Memory Trick

“Coins Rod Letters” 🪙✂️🔤

* Coin Change → fewest coins

* Rod Cutting → max money

* LCS → common letters
